

Big Data Analysis

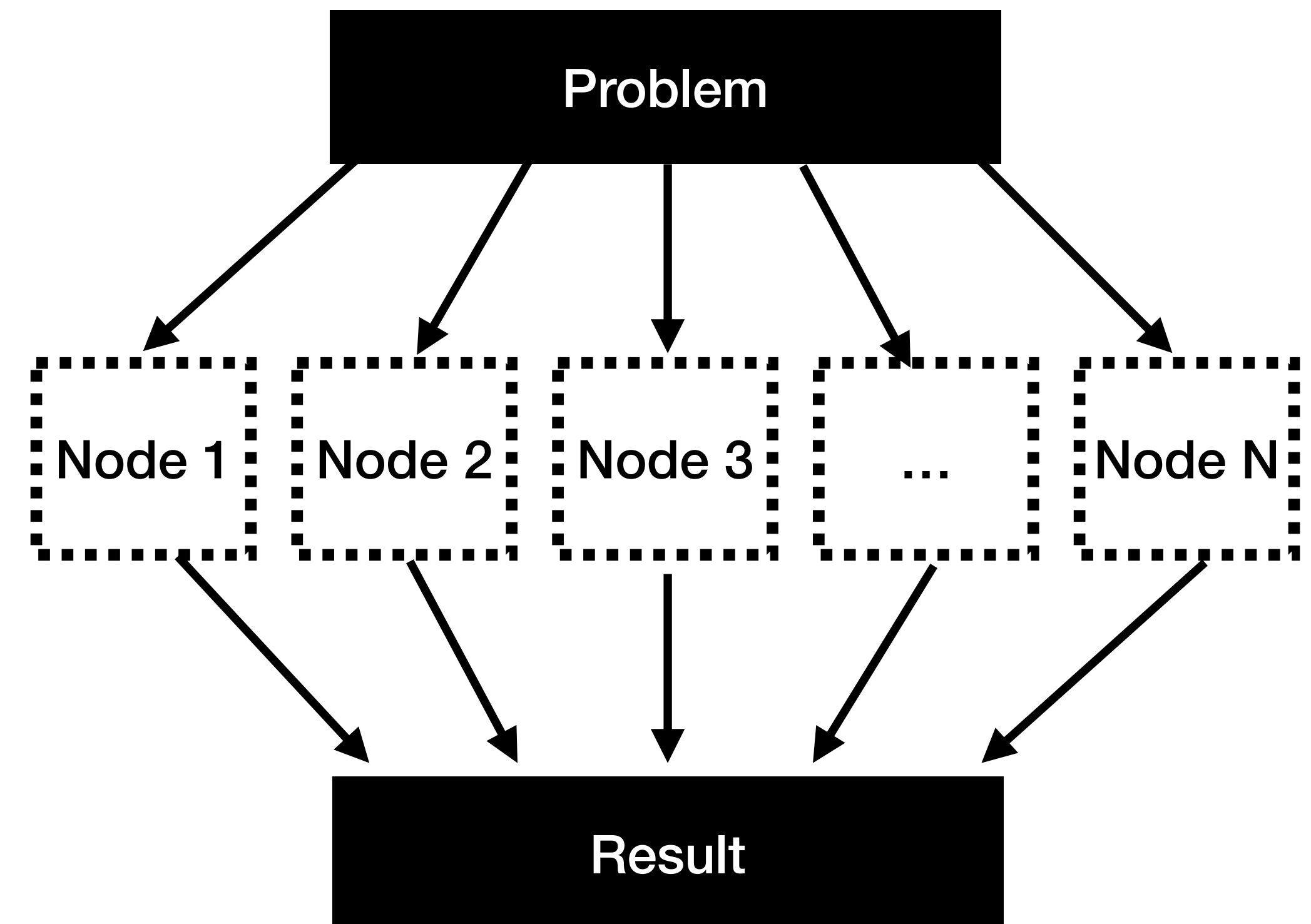
(Análise de Dados de Grandes Dimensões)

Distributed Computing and Processing Paradigms

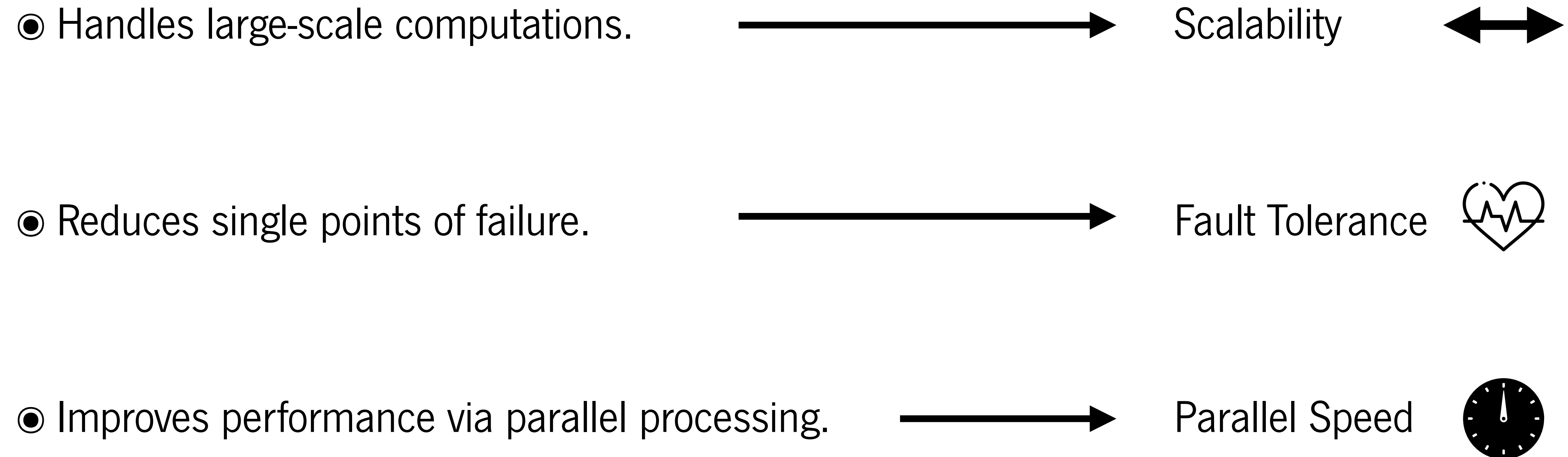


Introduction to Distributed Computing

- **Definition:** A system where multiple computers work together to solve problems.
- **Importance:** Scalability, reliability, and performance.
- **Examples:** Cloud services, AI models, financial transactions.



Why Distributed Computing?



Distributed Computing Paradigms

How we organize systems to distribute compute and data

Where compute happens (placement)

- Client-Server
- Peer-to-Peer (P2P)
- Edge & Fog Computing
- Cloud Computing

How it is organised (architecture)

- Cluster Computing
- Supercomputing
- Microservices
- Data-parallel processing (MapReduce / Spark)

Client-Server Model

- **Functionality** and **data** are **in the server**

- A **stub** runs embedded in the client

- provides an API to interact with the server
- abstracts the details of the protocol
- It is part of the server software package

- Example: the Web

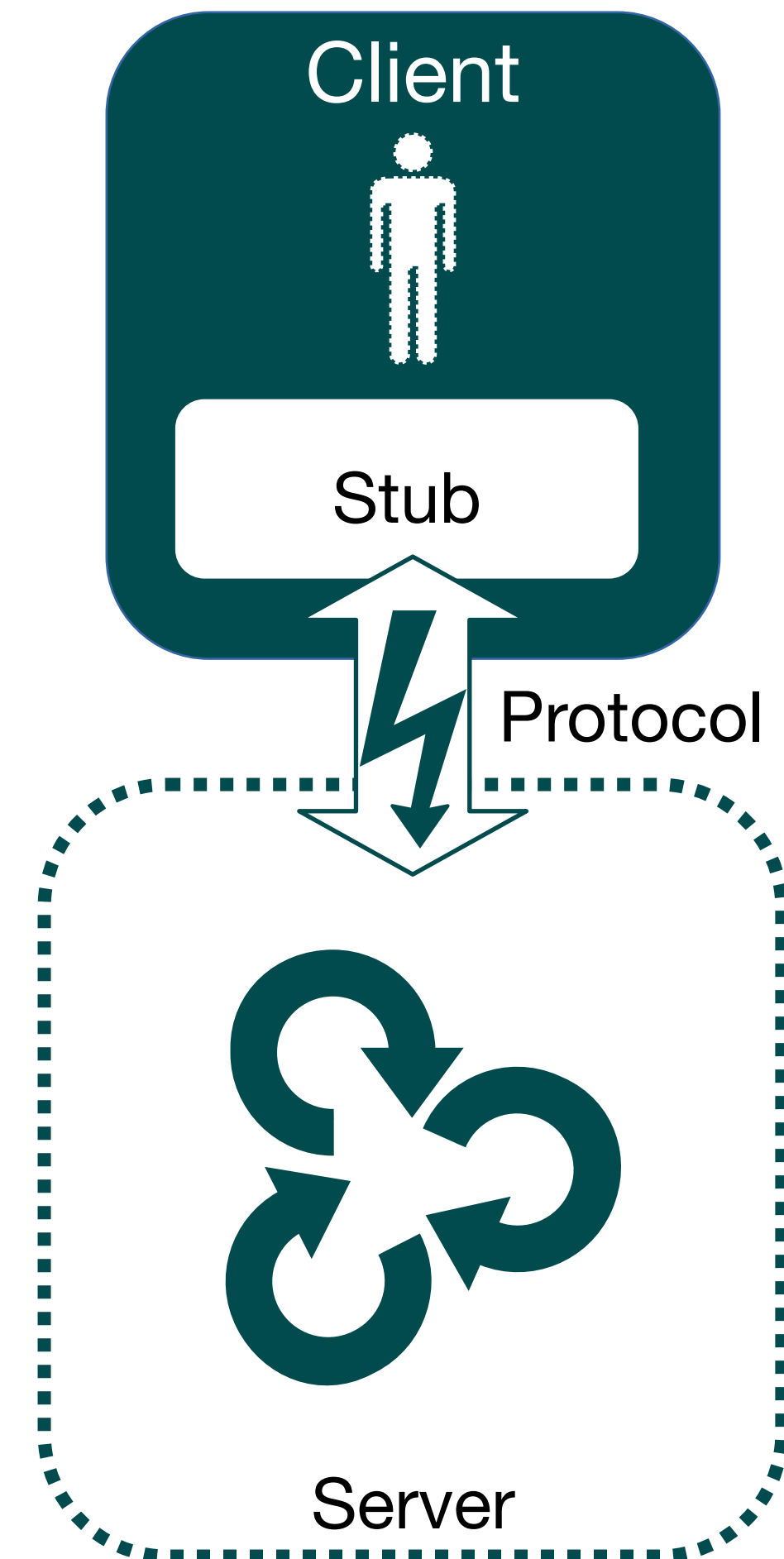
- “protocol” is HTTP

- **Pros**

- Simple management, centralized security

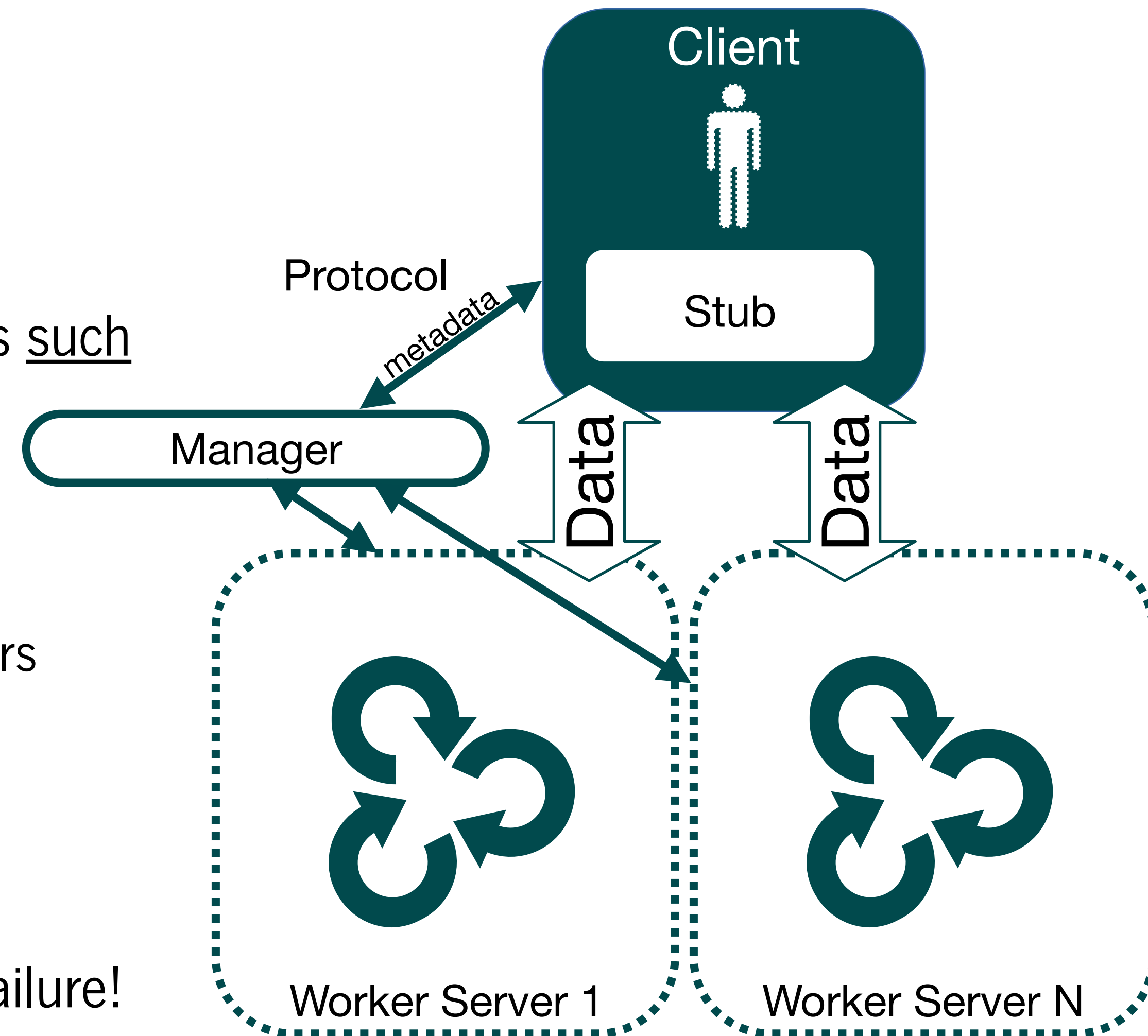
- **Cons**

- Bottleneck, single point of failure



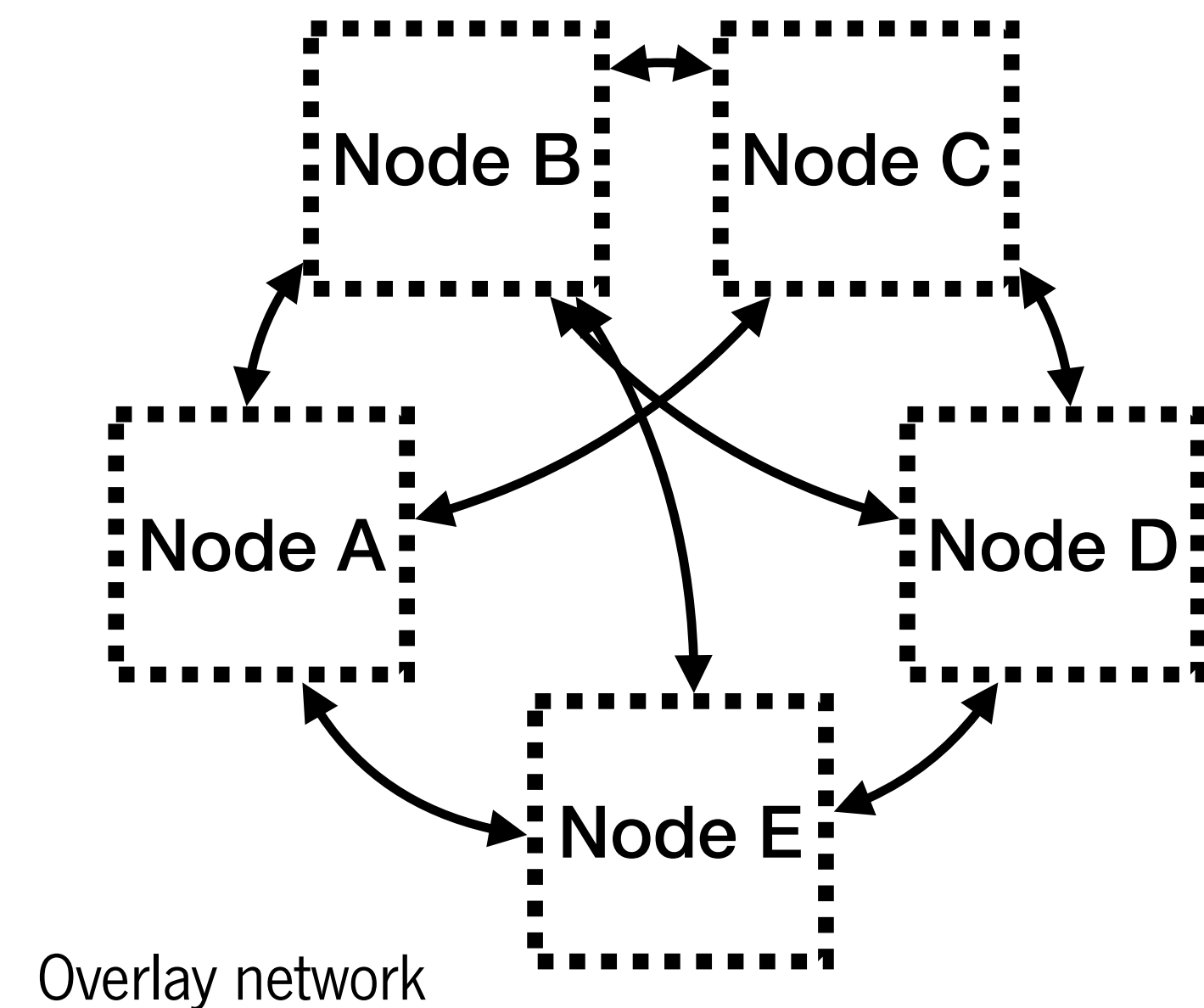
(Extra) Manager-Worker

- Functionality is split between a **manager** and **worker** servers
- Common architecture, for example, for storage systems such as HDFS, Lustre, ...
 - ▶ **Manager:** handles the location of files (metadata)
 - ▶ **Workers:** handle the content of files (data)
 - ▶ Data may be **replicated/sharded** across several workers
- Further addresses **scale-out** i.e., by splitting functionality!
- Manager may still be a single point of contention and failure!



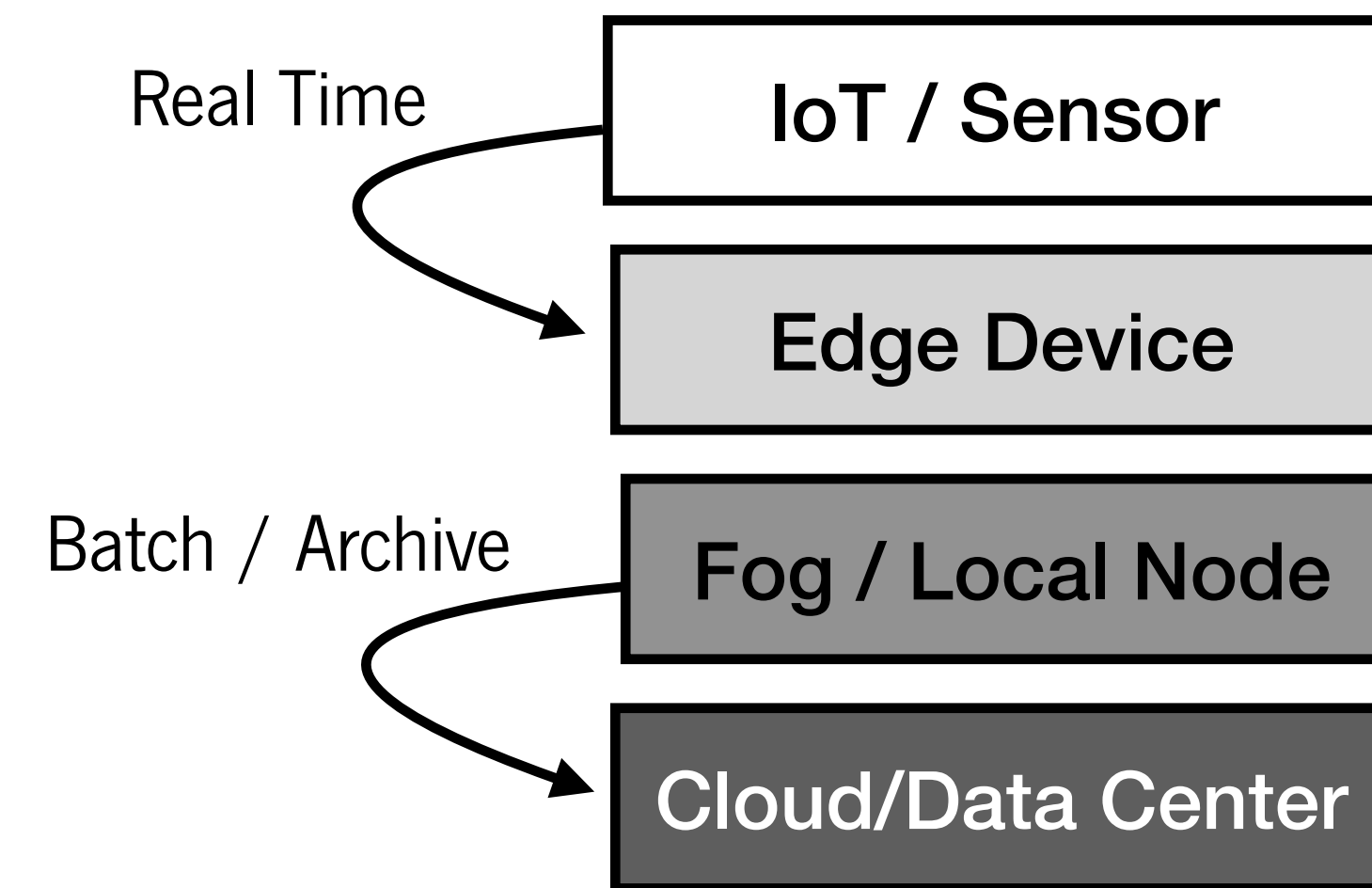
Peer-to-Peer (P2P) Model

- Nodes act as both clients and servers. As such, everyone can Request and Serve.
- Used for file sharing, content distribution, blockchain networks
 - E.g., BitTorrent, Blockchain, Skype.
- **Pros**
 - Decentralized, highly scalable, fault-tolerant (no single point of failure).
- **Cons**
 - Security risks, coordination overhead



Edge & Fog Computing

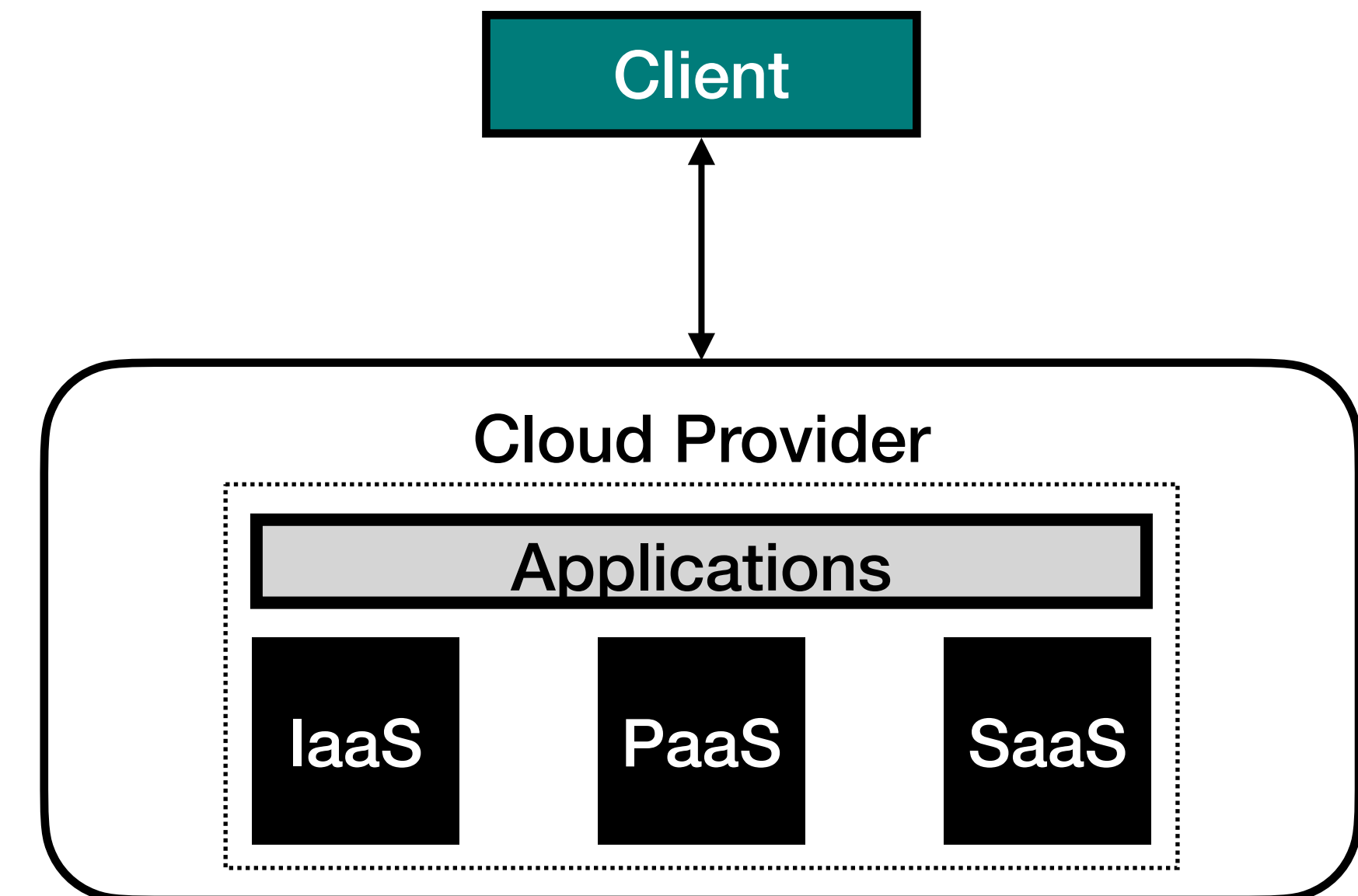
- Processing data closer to where it is generated (before reaching the cloud).
 - The main goal is to reduce latency and bandwidth
- Used for IoT devices, Smart Buildings, Smart Cities.
- **Pros:** Low latency, reduced bandwidth usage, real-time responses.
- **Cons:** Security challenges, limited local resources.



Some processing happens before data reaches the cloud.

Cloud Computing

- On-demand computing over the internet.
- **Types:** IaaS, PaaS, SaaS.
- **Examples:** AWS, Google Cloud, Microsoft Azure.
- **Pros:** Scalable, fast provisioning, global availability
- **Cons:** Security risks, dependency on providers (e.g., Google, AWS).



Cluster Computing

- **Multiple computers** connected to work as a single system.

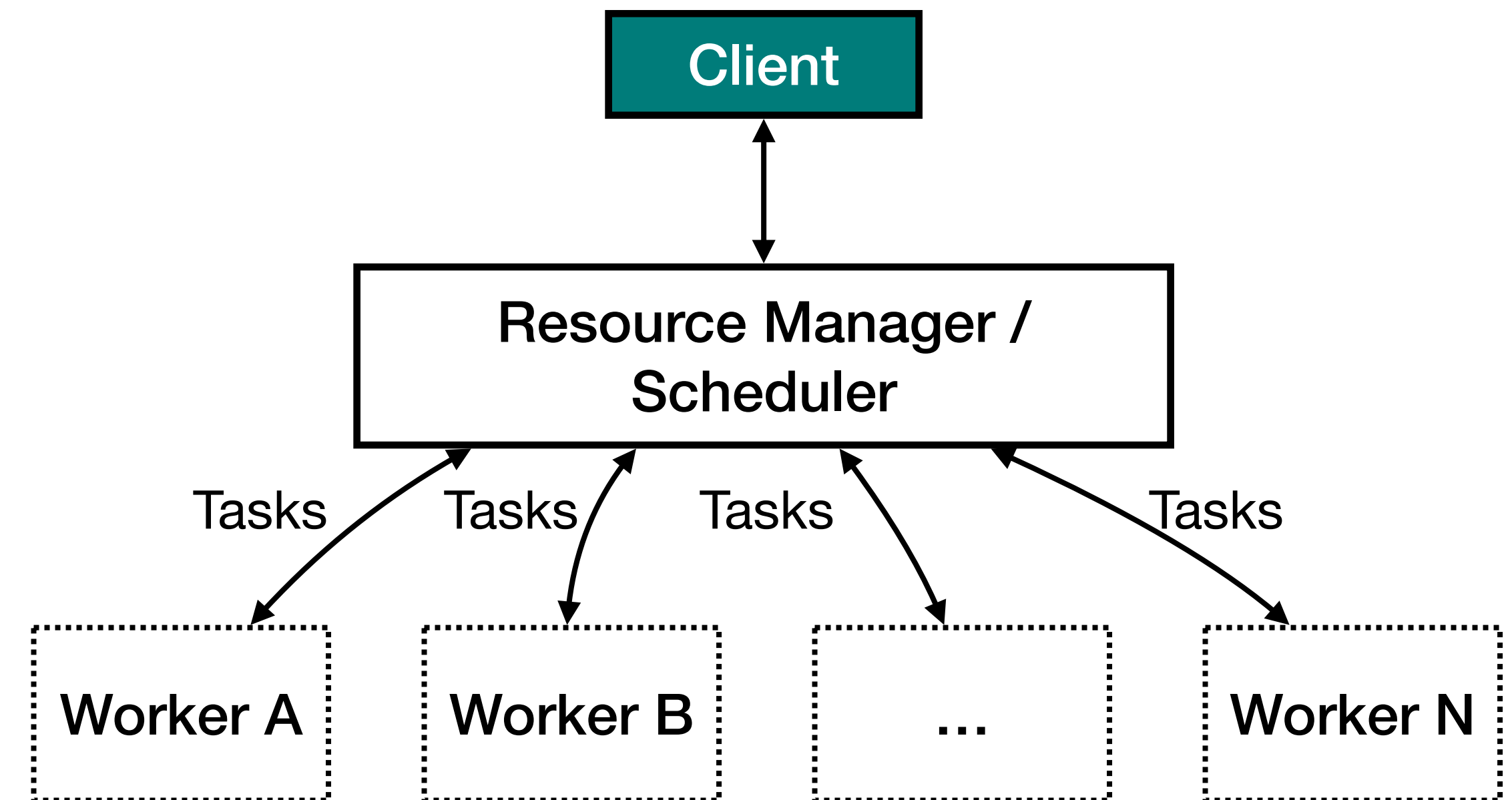
- The key idea is based on the parallel execution of tasks managed by a scheduler.

- Used for analytics, scientific jobs, large-scale services.

- Apache Spark, Beowulf clusters.

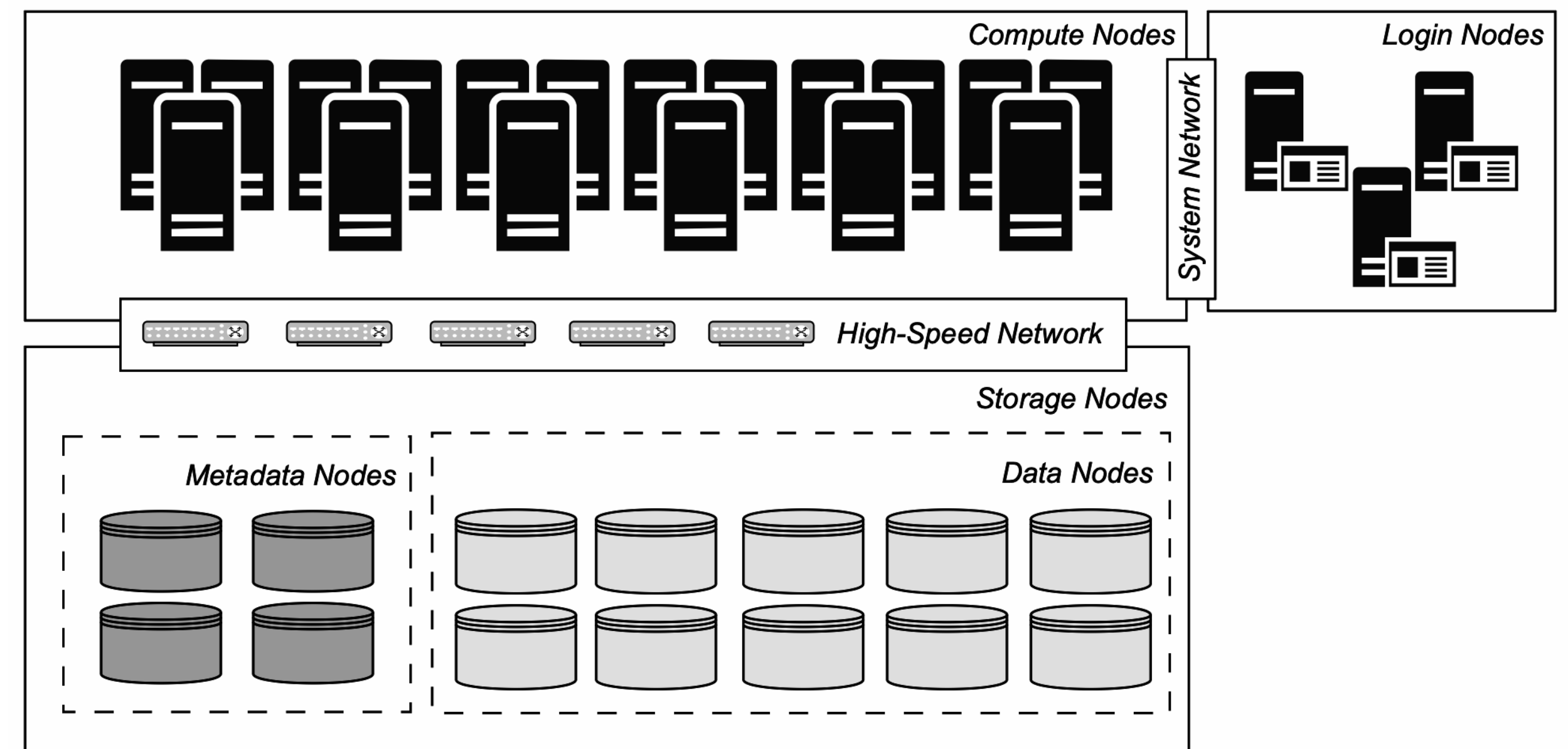
- **Pros:** High performance/throughput, cost-effective.

- **Cons:** Requires specialized setup, power consumption, network bottlenecks.



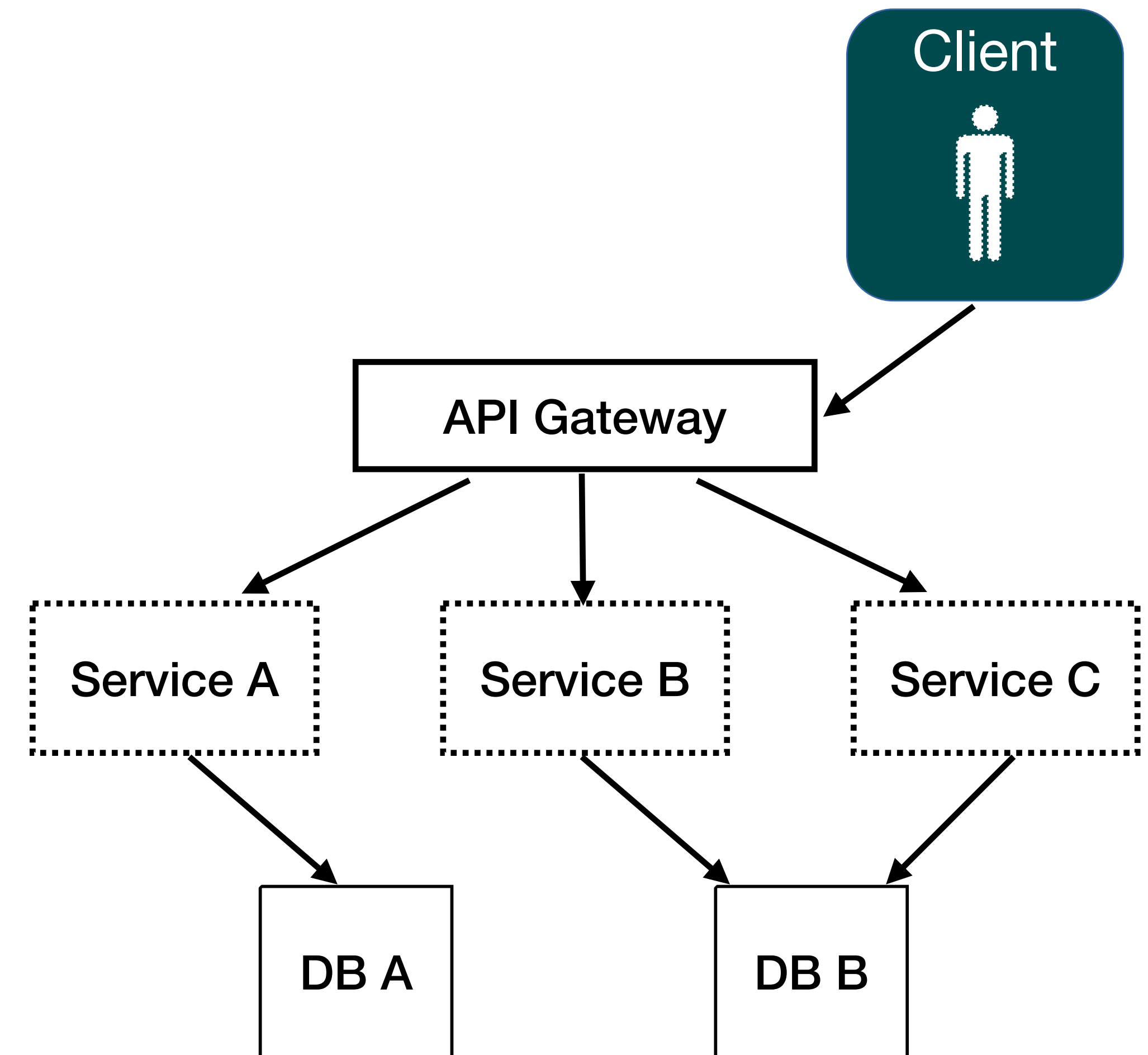
Supercomputing

- Extremely powerful computing for complex tasks.
 - It offers massive parallelism and high-speed interconnect.
- **Examples:** Fugaku, Deucalion, MN5, Aurora, LUMI, Leonardo
- **Applications:** Weather forecasting, scientific simulations, AI training
- **Pros:** Unmatched/maximum performance
- **Cons:** High energy and maintenance cost



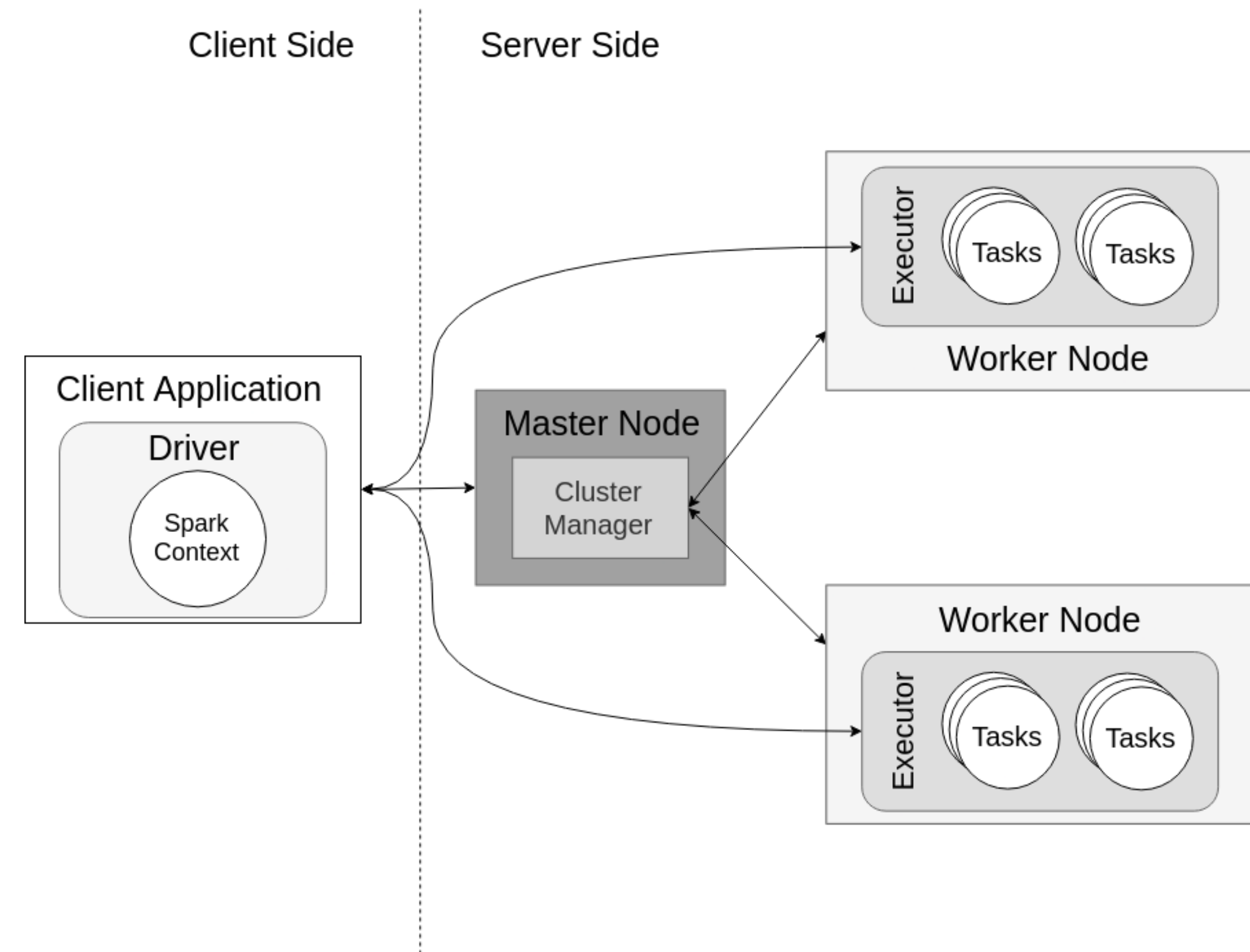
Microservices Architecture

- Applications are broken into small, independent services.
 - The main idea focus on the fact that services deploy/scale independently
- Used for large web platforms (e-commerce, streaming, fintech)
 - Netflix, Uber, Amazon.
- **Pros:** Scalable, modular.
- **Cons:** Increased complexity, requires orchestration and eventually coordination.



MapReduce & Data Parallelism

- Divides large datasets into smaller tasks for parallel processing.
- **Example:** Hadoop, Apache Spark
- **Pros:** Efficient for big data
- **Cons:** Requires distributed storage, complexity



Challenges in Distributed Computing

- **Latency and bandwidth:** messages are slow/limited compared to local calls
- **Fault tolerance:** machines/networks fail → retries/replication needed
- **Consistency:** replicas may disagree → coordination required
- **Security:** more nodes + more endpoints → bigger attack surface
- **Operational complexity:** deployment, monitoring, and upgrades are harder

Important notes

- Note that, distribution **patterns**, **mechanisms** and **architectures** can be combined to meet different performance, scale, and dependability requirements

Introduction to Distributed Processing

◎ **Definition:**

- Divide a computational task across multiple interconnected nodes.
- Nodes work in parallel, communicating over a network.

◎ **Importance:**

- **Scalability:** Easily add resources as demand grows.
- **Fault Tolerance:** System continues to operate despite individual node failures.
- **Performance:** Parallelism reduces processing time for complex tasks.
- **Resource Optimization:** Maximizes utilization of available computing power.

Examples (1)

● **Google Search & Indexing:**

- Uses clusters of servers to crawl, index, and serve search results across billions of web pages.
- Employs distributed file systems and data processing frameworks (like MapReduce) to handle massive datasets.

● **Netflix:**

- Leverages microservices architecture distributed across multiple data centers.
- Uses distributed processing for content recommendation, streaming optimization, and real-time analytics.

● **Amazon Web Services (AWS):**

- Offers distributed computing solutions (EC2, S3, DynamoDB) that power applications across various industries.
- Supports scalable, fault-tolerant, and high-performance processing for enterprise applications.

Examples (2)

● **Hadoop Ecosystem:**

- ▶ Widely adopted in industries such as finance, healthcare, and retail for big data analytics.
- ▶ Processes large datasets in parallel using MapReduce and related tools.

● **Scientific Research Projects:**

- ▶ SETI@home & Folding@home: Utilize volunteer computing across thousands of personal devices.
- ▶ Solve complex problems like protein folding or analyzing astronomical data through distributed processing.

Challenges of Big Data Processing

● Volume:

- Managing and storing massive amounts of data.

● Velocity:

- Processing data at high speeds, often in real-time.

● Variety:

- Handling multiple data formats (structured, unstructured, semi-structured).

❖ Veracity:

- Ensuring data quality and reliability.

❖ Scalability:

- Maintaining performance as data and system complexity increase.

❖ Infrastructure Complexity:

- Coordinating distributed storage, computation, and network resources.

❖ Security & Privacy:

- Protecting sensitive data across distributed environments.

MapReduce

● **A Paradigm for Large-Scale Data Processing**

● **Definition:**

- ▶ A programming model for processing vast amounts of data in parallel.
- ▶ Developed by Google (2004) to handle large-scale data processing tasks.

● **Purpose:**

- ▶ Simplify distributed computing by breaking down tasks into two primary functions: Map and Reduce.

Core Components of MapReduce

● Map Function

- ▶ Processes input key-value pairs.
- ▶ Generates intermediate key-value pairs.

● Shuffle & Sort

- ▶ Automatically groups intermediate data by keys.
- ▶ Prepares data for the Reduce phase.

● Reduce Function:

- ▶ Aggregates and processes grouped data.
- ▶ Produces final output results.

MapReduce Workflow

- Data Splitting:
 - Input data is divided into manageable chunks.
- Mapping Phase:
 - Each chunk is processed in parallel by the Map function.
- Shuffling & Sorting:
 - Intermediate data is grouped based on keys.
- Reducing Phase:
 - The Reduce function aggregates the results to produce the final output.
- Output
 - Consolidated data is written to storage.

Advantages of MapReduce

- Scalability:
 - Easily scales across clusters of commodity hardware.
- Fault Tolerance:
 - Automatically handles node failures.
- Simplicity:
 - Abstracts complex parallel processing, enabling developers to focus on core logic.
- Cost-Effective:
 - Leverages distributed resources for large-scale data processing.

Limitations and Challenges

- Latency:

- ▶ Not ideal for real-time processing due to its batch-oriented nature.

- I/O Intensive:

- ▶ Intermediate data is written to disk, which can slow down processing.

- Complex Workflows:

- ▶ Handling iterative or stateful computations can be challenging.

- Debugging:

- ▶ Distributed nature may complicate error detection and debugging.

Real-World Applications

- Search Engines:

- ▶ Indexing and ranking billions of web pages (e.g., Google).

- Log Analysis:

- ▶ Processing server logs for insights and security monitoring.

- Data Mining & Analytics:

- ▶ Analyzing large datasets for patterns and trends.

- Scientific Research:

- ▶ Handling data from simulations and experiments (e.g., genomics).

MapReduce in the Ecosystem

- Apache Hadoop:

- ▶ Popular open-source implementation of MapReduce.

- Complementary Tools:

- ▶ Pig & Hive:

- Simplify writing and managing MapReduce jobs.

- ▶ HBase:

- NoSQL database built on top of Hadoop.

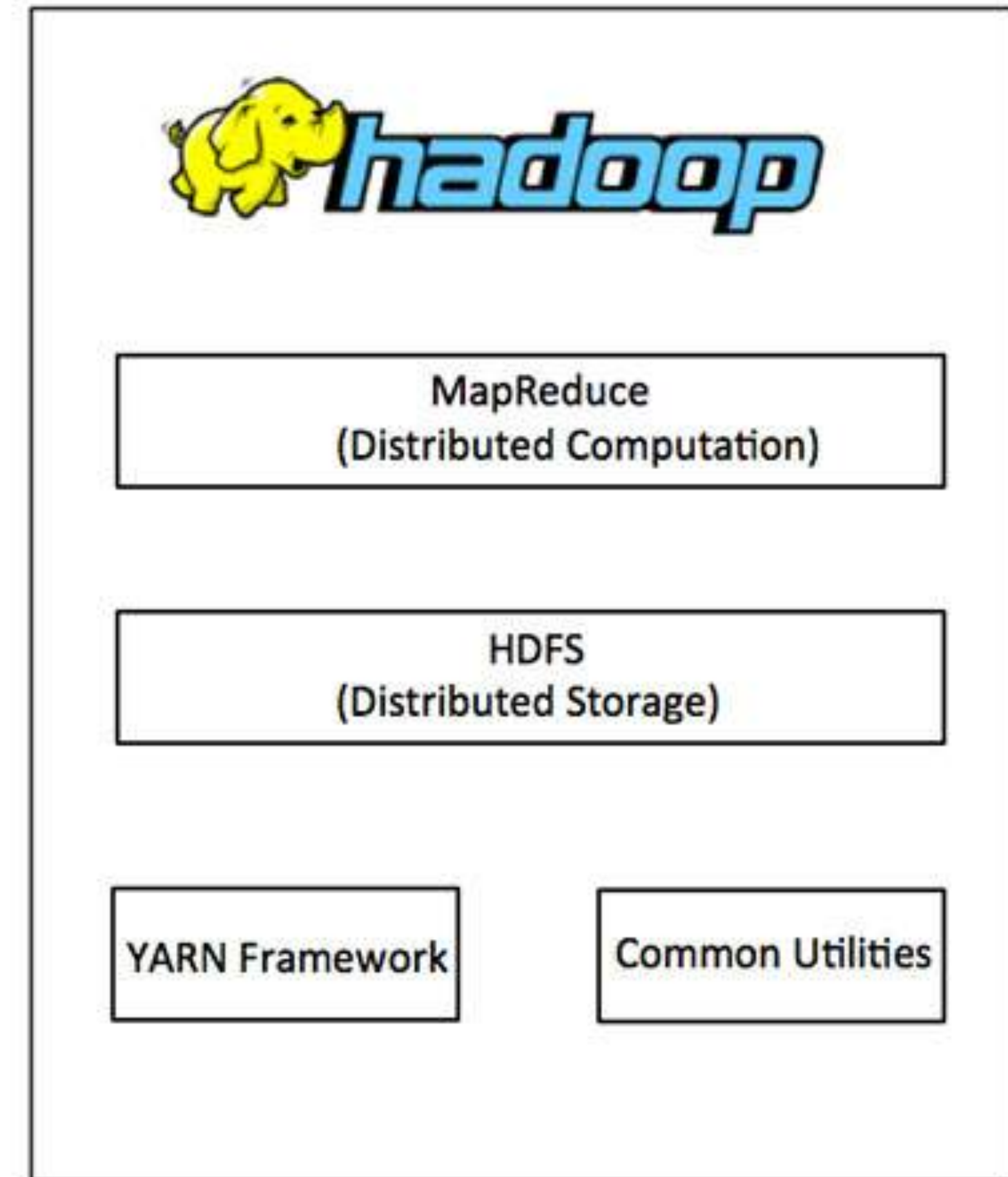
- Evolution:

- ▶ Spark and other frameworks build on MapReduce principles for enhanced performance.

Hadoop Architecture

● Components

- ▶ HDFS for storage
- ▶ MapReduce for processing
- ▶ YARN for resource Management



Fonte: https://www.tutorialspoint.com/hadoop/images/hadoop_architecture.jpg

HDFS Architecture

- Master-Slave Architecture:

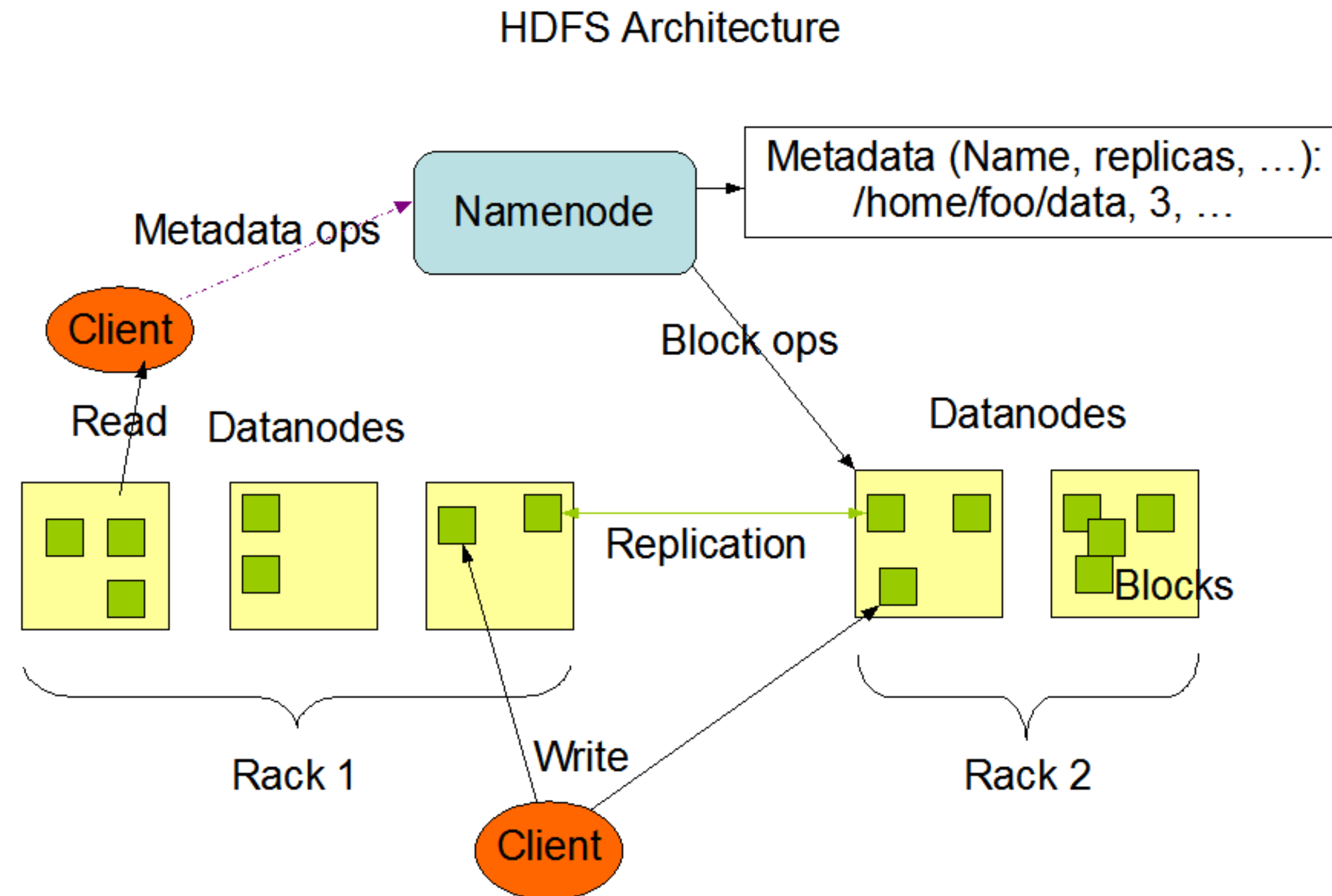
- ▶ NameNode:
 - Centralized metadata management
- ▶ DataNodes:
 - Store and retrieve actual data blocks

- Secondary NameNode:

- ▶ Assists with checkpointing (not a hot standby for NameNode)

- Data Blocks:

- ▶ Files are split into fixed-size blocks (default 128MB or configurable)



Fonte: <https://hadoop.apache.org/docs/r1.2.1/images/hdfsarchitecture.gif>

HDFS Components

- NameNode:

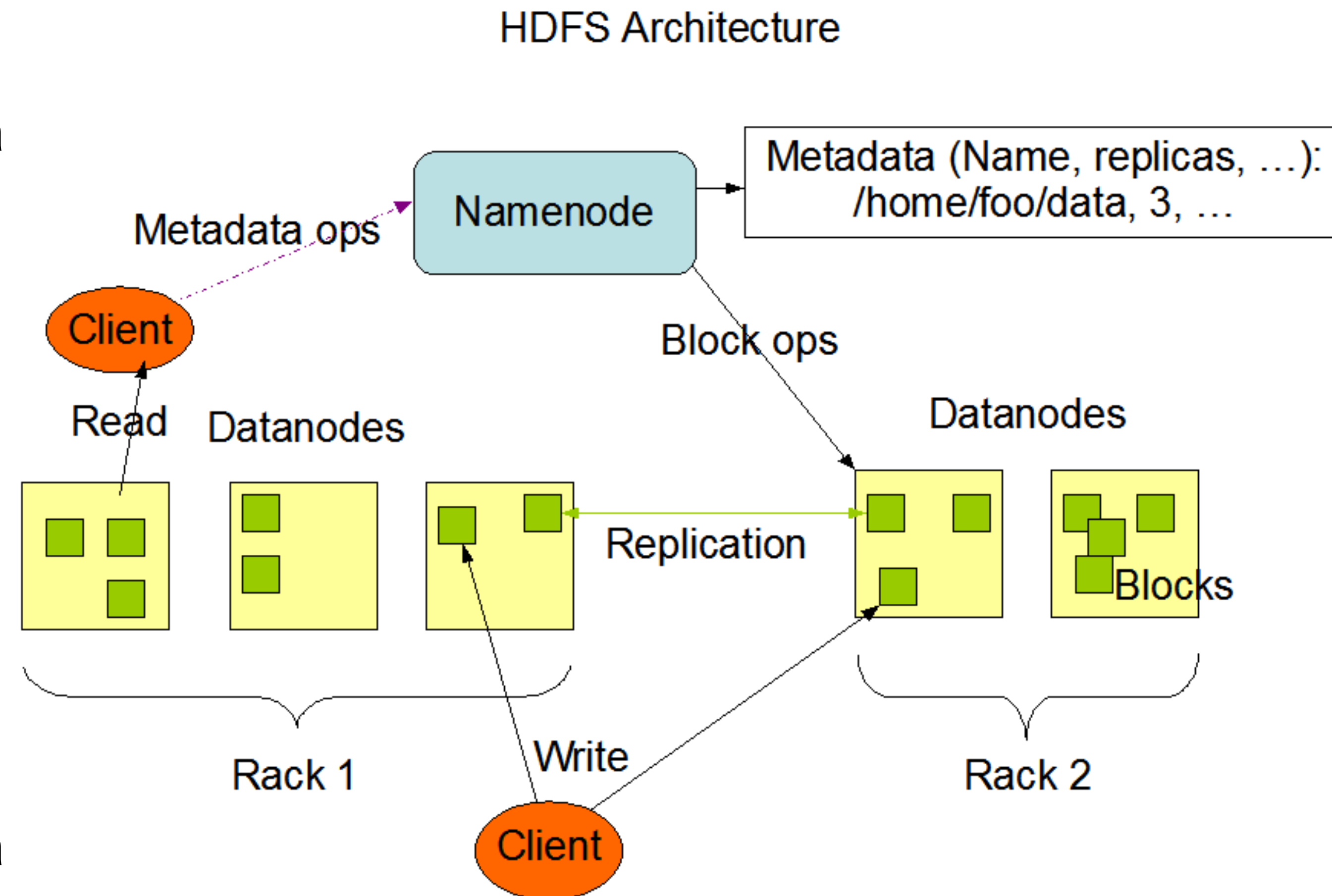
- ▶ Maintains the directory tree and metadata
- ▶ Manages file-to-block mapping and replication policies

- DataNodes:

- ▶ Handle storage, retrieval, and replication of data blocks
- ▶ Report regularly to the NameNode

- Block Replication:

- ▶ Ensures fault tolerance by replicating data (default replication factor: 3)



HDFS Data Flow & Operation

HDFS Architecture

- Write Operation:

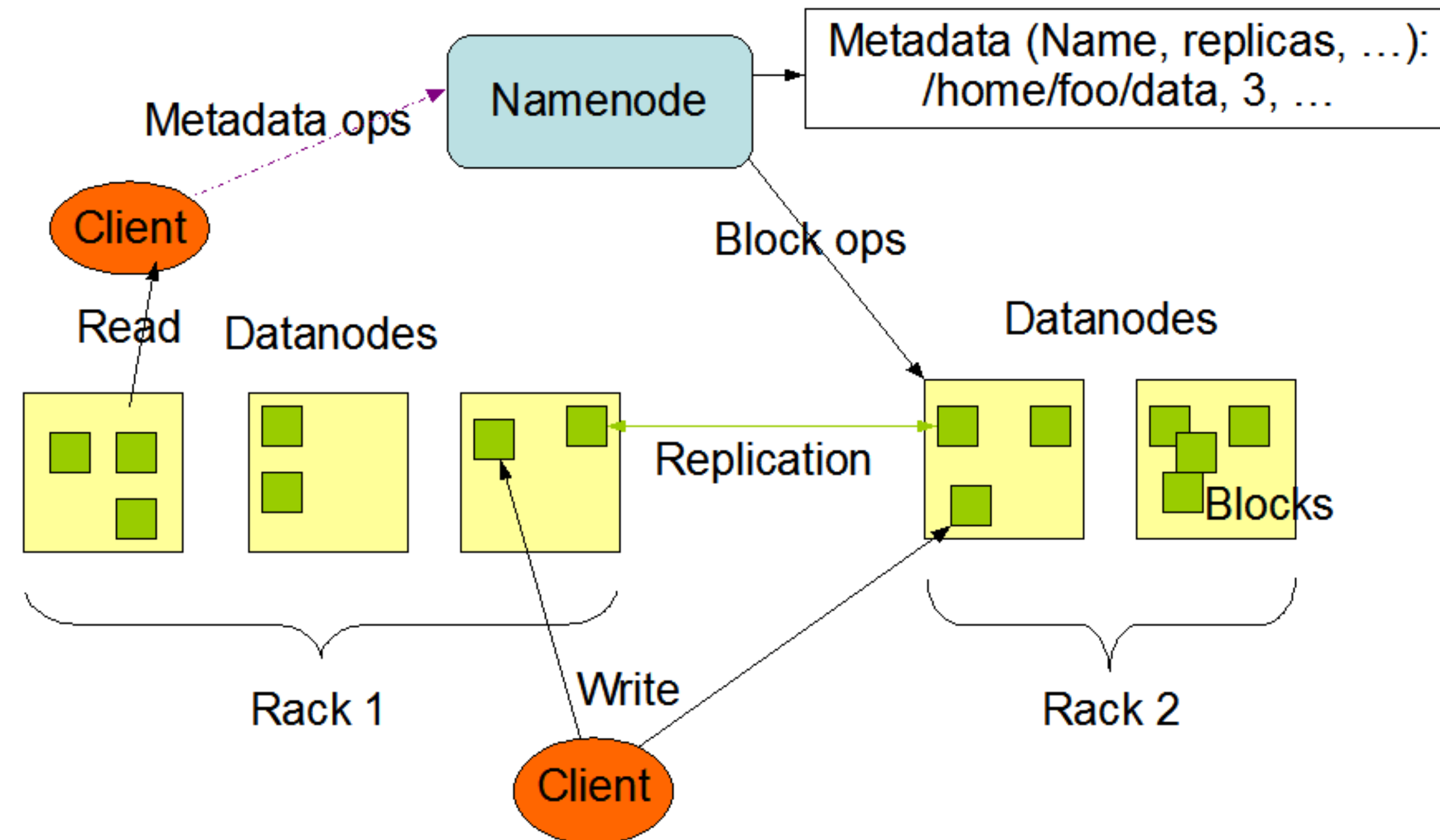
- ▶ Client contacts NameNode to get block locations
- ▶ Data is written sequentially across DataNodes

- Read Operation:

- ▶ Client fetches metadata from NameNode
- ▶ Data is read directly from DataNodes

- Fault Tolerance:

- ▶ Automatic re-replication on node failure



Advantages of Using HDFS

- Scalability:
 - ▶ Easily scales out by adding more DataNodes
- High Throughput:
 - ▶ Optimized for batch processing rather than low-latency access
- Fault Tolerance:
 - ▶ Data replication and self-healing mechanisms
- Integration:
 - ▶ Seamlessly works with other Hadoop ecosystem components

Use Cases

- Big Data Analytics:
 - ▶ Storing and processing large datasets
- Data Warehousing:
 - ▶ Aggregating large volumes of log and transactional data
- Real-Time Processing:
 - ▶ Supporting tools like Apache Spark for in-memory computations
- Machine Learning:
 - ▶ Providing data storage for training models

Challenges & Limitations

- NameNode as a Single Point of Failure:
 - ▶ Mitigated by high availability setups, but historically a concern
- Small File Problem:
 - ▶ Inefficiencies when storing a large number of small files
- Network Dependency:
 - ▶ High network usage during replication and rebalancing

MapReduce Framework

● Concept & Workflow:

- Parallel data processing model divided into Map and Reduce tasks

● Phases:

- Map Phase:
 - Processes input data into key-value pairs

● Shuffle & Sort:

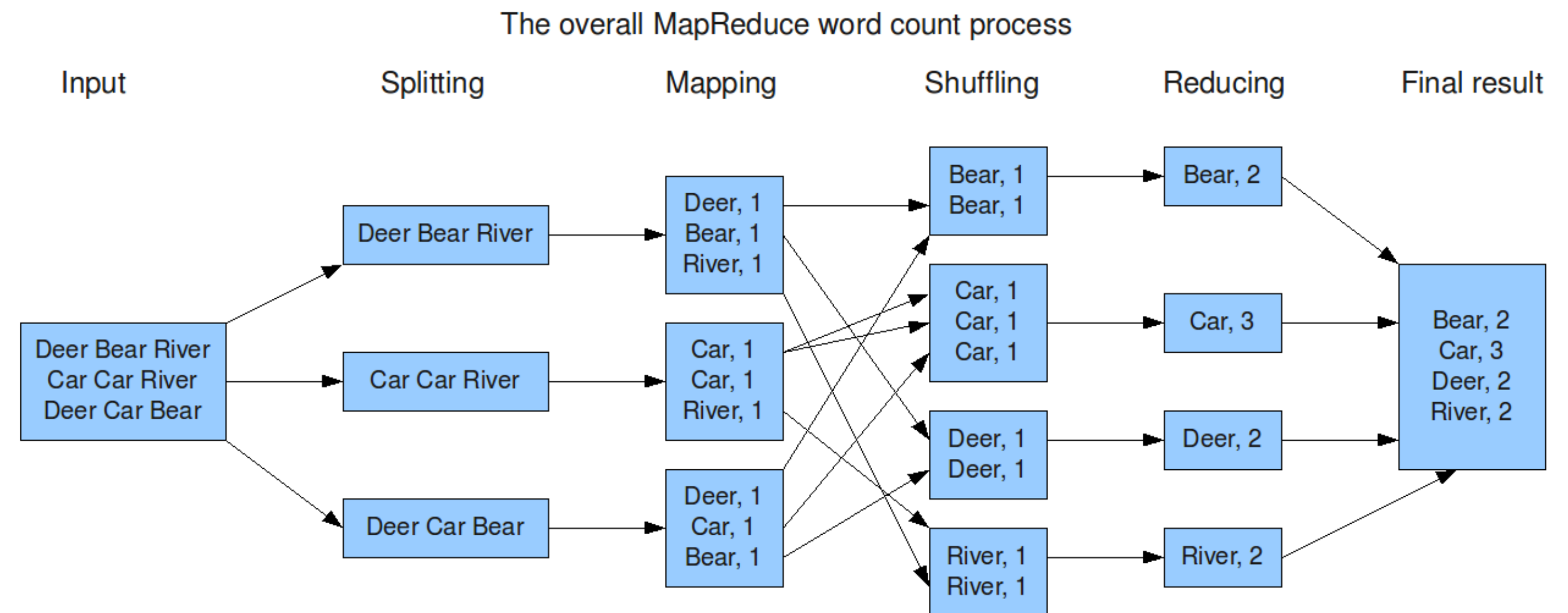
- Organizes data for the reduce tasks

● Reduce Phase:

- Aggregates and produces final results

● Advantages & Limitations:

- Scalability and fault tolerance versus potential latency in batch processing



Fonte: <https://www.todaysoftmag.com/images/articles/tsm33/large/a11.png>

YARN (Yet Another Resource Negotiator)

● Decouples resource management from processing, improving cluster utilization.

● Key Components:

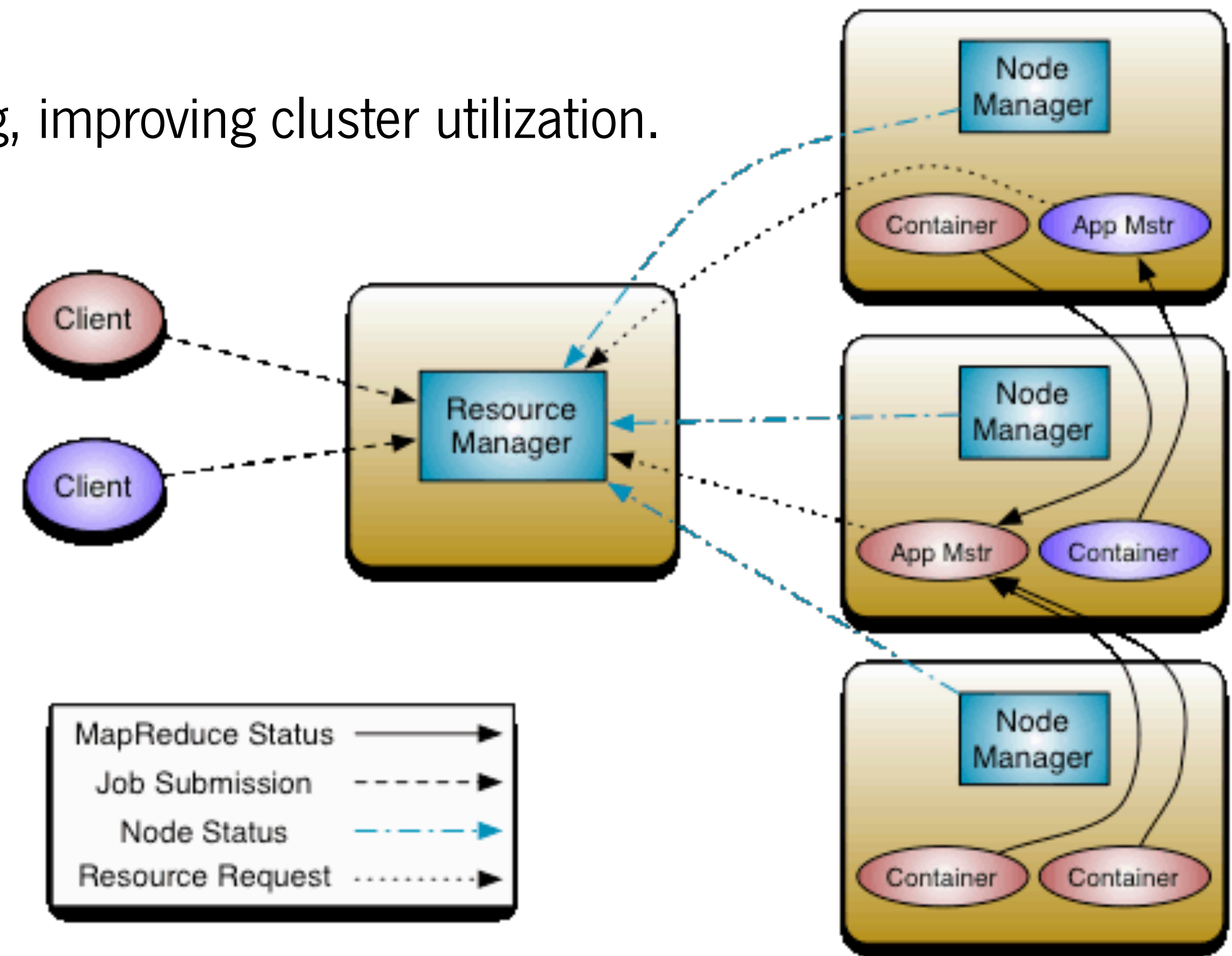
- Resource Manager:
 - Global cluster resource allocation.
- Node Manager:
 - Manages individual node resources.

● Application Master:

- Orchestrates task execution for applications.

● Impact on Hadoop:

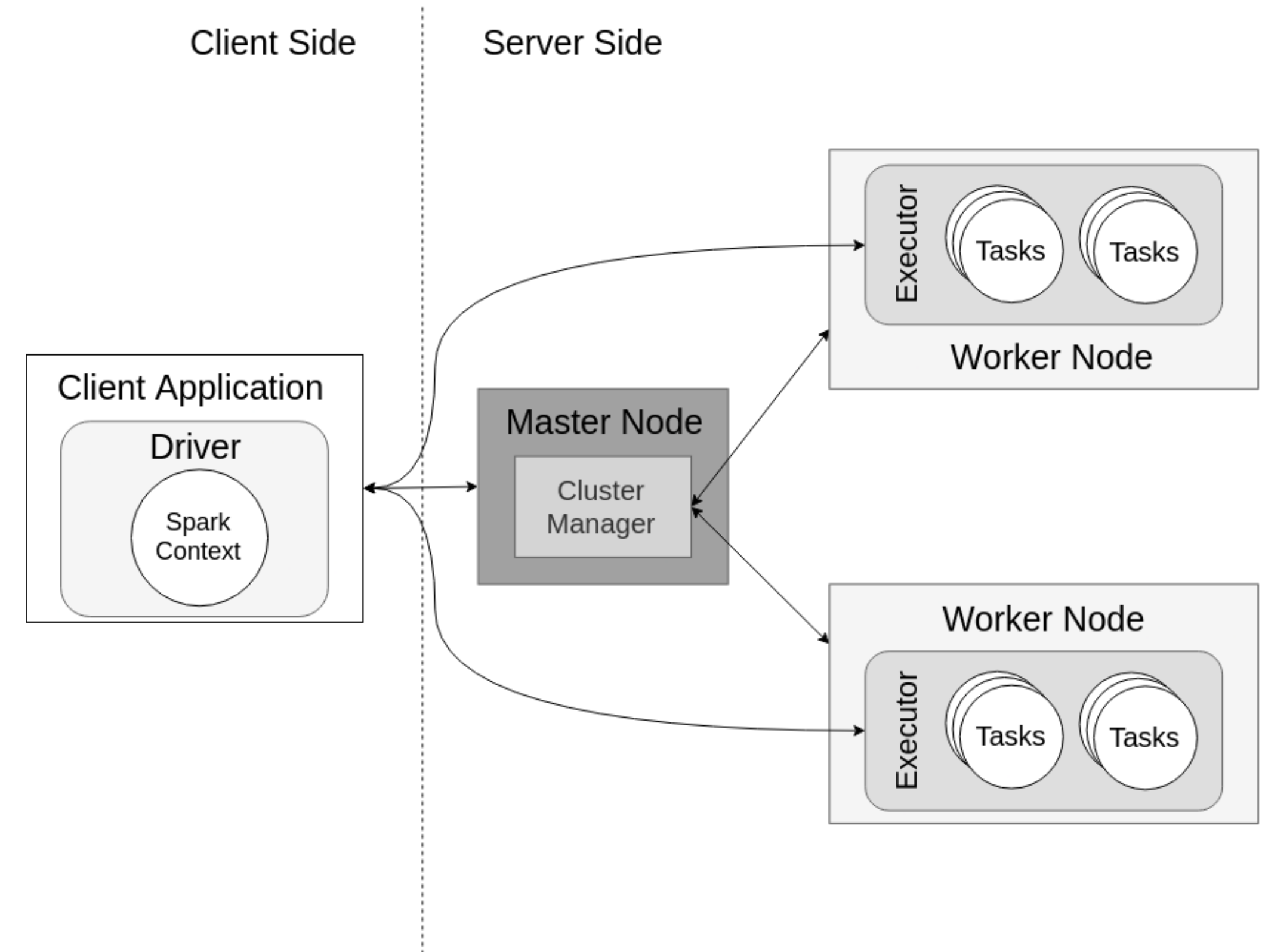
- Enhanced flexibility and multi-tenancy within the cluster.



Fonte: https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/yarn_architecture.gif

Apache Spark

- A unified analytics engine for large-scale data processing.
- Provides fast in-memory processing and supports diverse workloads.
- Widely used for batch processing, streaming, machine learning, and graph processing.



Spark core Components

● Spark Driver

- ❖ Process to execute user apps
- ❖ SparkContext schedules jobs execution and negotiate with cluster manager

● Cluster Manager

- ❖ Mesos; YARN; Spark Standalone

● Worker Nodes / Executors

- ❖ Run tasks scheduled by the driver
- ❖ Store computation results in memory, on disk or off-heap
- ❖ Interact with storage systems

Fonte: <https://spark.apache.org/docs/latest/img/cluster-overview.png>

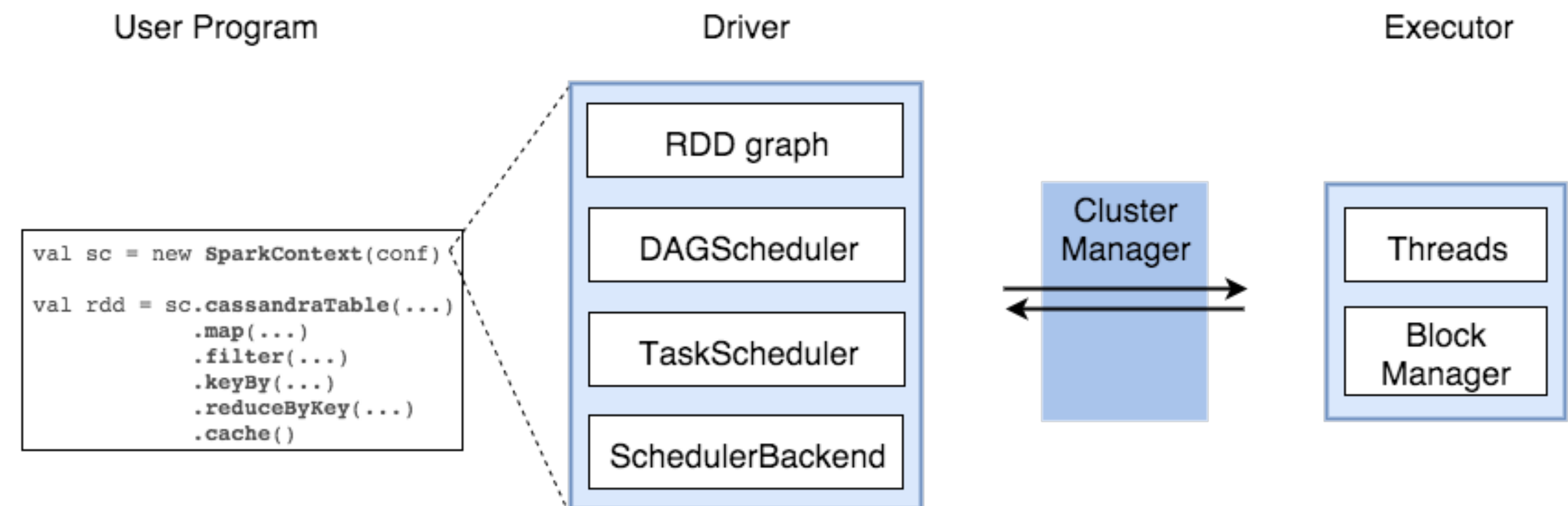
Spark core Components

● RDDs (Resilient Distributed Datasets):

❖ Immutable distributed collections.

● DAG Scheduler:

❖ Translates jobs into stages and tasks.



Fonte: <https://datastrophic.io/blog/2016-03-03/spark-components.png>

Questions?